How to Model Dynamical Systems

Modellierung dynamischer Systeme

Wissenschaftliche Arbeit im Rahmen der Forschungspraxis
an der Fakultät für Elektrotechnik und Informationstechnik der
Technischen Universität München.

Betreut von	Univ.-Prof. Dr.-Ing. Sandra Hirche Max Beier Lehrstuhl für Informationstechnische Regelung
Eingereicht von	Aleksandar Marjanovic Heiglhofstraße 64 Munich +491708272972
Eingereicht am	München, den 01.04.2025

TECHNISCHE UNIVERSITÄT MÜNCHEN
LEHRSTUHL FÜR INFORMATIONSTECHNISCHE REGELUNG
ORDINARIA: UNIV.-PROF. DR.-ING. SANDRA HIRCHE

September 27, 2024

MASTER'S THESIS

How to Model Dynamical Systems

Problem description:

Many machine-learning problems related to physics involve data in the form of time series. Consequently, a zoo of modeling techniques for dealing with time series data exists. However, most of these techniques are built behaviorally and rely on handcrafted and ad-hoc features or powerful architectures that neglect the specific temporal structure (CNN, transformer, SSM). All methods have strengths and weaknesses and perform well on different tasks (classification, regression) and applications. This makes statements about when to use which method vague and makes model search and engineering hard. A prime example is the tensorflow time series forecasting tutorial [1].
This work uses a system theoretic approach to compare different model classes, for example, models relying on differential equations in state-space [3], sequence-2-sequence [4], or operator learning [2]. It will curate a collection of datasets (synthetic and real-world) and put them in the taxonomy of system theory. On this basis, it will explore patterns of models that apply to different kinds of tasks.

Method: Datasets will be curated based on system theoretic properties. The model set will be chosen from existing implementations in Python. An experimental pipeline will be set up to evaluate models.

Goal: Find out how and if system theoretic classifications of times series as dynamical systems can help choose appropriate models.

Outcome: A report that gives people new to time series and dynamical systems an overview of existing models and system theoretic intuition about which model is likely to work with which kind of data.

Tasks:

- Literature research on Methods and Datasets
- Conception of a Comparison Study
- Implementation
- Evaluation of Results

Bibliography:

[1] Time series forecasting | TensorFlow Core.
[2] Petar Bevanda, Max Beier, Armin Lederer, Alexandre Capone, Stefan Georg Sosnowski, and Sandra Hirche. Gaussian Process-Based Representation Learning via Timeseries Symmetries. In *ICML 2024 Workshop on Geometry-grounded Representation Learning and Generative Modeling*, June 2024.
[3] Patrick Kidger. On Neural Differential Equations, February 2022. arXiv:2202.02435 [cs, math, stat].
[4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. Technical Report arXiv:1706.03762, arXiv, December 2017. arXiv:1706.03762 [cs] type: article.

Supervisor:	M. Sc. Max Beier
Start:	XX.XX.2024
Intermediate Report:	XX.XX.2024
Delivery:	XX.XX.2024

(S. Hirche)
Univ.-Professor

Abstract

Viele maschinelle Lernprobleme beinhalten Daten in Form von Zeitreihen. Daher gibt es eine Vielzahl von Modellierungstechniken zur Verarbeitung von Zeitreihendaten. Die meisten dieser Techniken sind verhaltensbasiert und stützen sich entweder auf manuell erstellte Merkmale oder leistungsstarke Architekturen, die die spezifische zeitliche Struktur vernachlässigen (z. B. RNNs, Transformer, SSMs). Alle Methoden haben ihre Stärken und Schwächen und eignen sich für unterschiedliche Aufgaben (z. B. Klassifikation, Regression) und Anwendungen. Dadurch bleibt unklar, wann welche Methode verwendet werden sollte, was die Modellauswahl und -entwicklung erschwert. Diese Arbeit verfolgt einen systemtheoretischen Ansatz, um verschiedene Modellklassen zu vergleichen. Dafür wird eine Sammlung von Datensätzen (sowohl synthetisch als auch real) zusammengestellt und in die Taxonomie der Systemtheorie eingeordnet. Auf dieser Grundlage werden Muster in den Modellen untersucht, die für unterschiedliche Arten von Aufgaben besonders geeignet sind.

Inhaltsverzeichnis

1 Einführung **5**

2 Modellsätze **7**
2.1 Rekurrente Neuronale Netze 7
2.2 Transformer 8
2.3 Koopman Operator 11

3 Systemdatensätze zur Evaluierung der Modelle **15**
3.1 Van-der-Pol-Oszillator 15
3.2 Van-der-Pol-Oszillator mit Rauschen 15
3.3 Lorenz-System 16
3.4 Raumtemperatur-Messungen 16

4 Ergebnisse **17**
4.1 Implementierungskomplexität 17
4.2 Zeitaufwand und Testfehler 18

5 Fazit **21**

Abbildungsverzeichnis **23**

Tabellenverzeichnis **25**

Bibliography **27**

Kapitel 1

Einführung

Im Folgenden werden die Grundbegriffe erklärt und die Struktur der Arbeit näher erläutert. Eine Zeitreihe (englisch: time series) ist eine geordnete Sammlung von Messungen, die in regelmäßigen Abständen durchgeführt werden. Eine solche Zeitreihe entsteht, wenn ein dynamisches System beobachtet wird (z. B. Temperaturänderungen, Aktienkurse, Roboterbewegungen).Die Prognose einer solchen Zeitreihe umfasst die Analyse sequentieller Datenpunkte, um Trends, Muster und Abhängigkeiten aufzudecken, die zur Vorhersage zukünftiger Werte genutzt werden können. Die Anwendungsgebiete sind vielfältig und reichen von Börsen- und Energieverbrauchsprognosen über Wettervorhersagen bis hin zur Bedarfsplanung im Supply Chain Management. Der Einfluss von Zeitreihenanalysen auf den Markt nimmt mit der Weiterentwicklung des maschinellen Lernens stark zu. Unternehmen, die diese Prognosemodelle nutzen, bleiben nicht nur der Konkurrenz voraus, sondern setzen auch neue Maßstäbe in Effizienz, Innovation und Anpassungsfähigkeit in einem sich ständig weiterentwickelnden Markt. [Pat23]
Die Zeitreihenprognose nutzt eine Vielzahl von Methoden – von klassischen statistischen Techniken bis hin zu modernen Ansätzen des maschinellen Lernens. In dieser Arbeit werden RNNs, Transformer-Modelle und die Koopman-Analyse implementiert und an verschiedenen Datensätzen trainiert. Diese Datensätze werden teils synthetisch erstellt, mithilfe von Differentialgleichungen modelliert oder basieren auf Sensormessungen der Raumtemperatur.
In den folgenden Kapiteln werden die Charakteristik und Struktur der Modelle beschrieben, die Entscheidungslogik hinter der Auswahl der Datensätze erläutert und schließlich eine Schlussfolgerung gezogen, welches Modell die beste Prädiktion für ein bestimmtes dynamisches System liefert.

Kapitel 2

Modellsätze

Zur Untersuchung und Auswertung werden die folgenden Modelle ausgewählt. Jedes Modell wird anhand seiner Charakteristiken beschrieben, und seine Struktur wird näher erläutert. Für weitere Details wird auf die im Folgenden angegebenen Referenzen verwiesen.

2.1 Rekurrente Neuronale Netze

Die folgende Beschreibung basiert auf dem Vorlesungsskript Künstliche Neuronale Netze zur Identifikation mechatronischer Systeme [End24]. Rekurrente neuronale Netze (RNNs) sind künstliche neuronale Netze zur Verarbeitung von Zeitreihendaten und sequenziellen Informationen. Im Gegensatz zu herkömmlichen neuronalen Netzen verfügen RNNs über Speicherelemente und können aktuelle Eingaben anhand historischer Informationen auswerten. Dank dieser Eigenschaften sind RNNs besonders effektiv bei der Modellierung zeitlicher Abhängigkeiten und sequenzieller Beziehungen.RNNs verarbeiten aufeinanderfolgende Datenpunkte mithilfe von Schleifen. In jedem Schleifendurchlauf kombiniert das Netzwerk den vorherigen Zustand (frühere Einträge oder latente Zustände) mit der aktuellen Eingabe, um eine Ausgabe sowie den nächsten Zustand zu erzeugen. Dieser Zustand wird im darauffolgenden Zeitschritt weiterverwendet, wodurch das Netzwerk langfristige Abhängigkeiten in den Daten erfassen kann.Eine zentrale Komponente von RNNs ist die Speicherzelle, auch latenter Zustand genannt. Dieser latente Zustand speichert Informationen aus vorherigen Zeitschritten und wird mit der aktuellen Eingabe kombiniert, um eine neue Ausgabe sowie einen aktualisierten latenten Zustand zu berechnen. Auf diese Weise kann das RNN nicht nur aktuelle Eingaben, sondern auch frühere Informationen berücksichtigen.

$$h_t = \tanh(x_t W_{ih}^T + b_{ih} + h_{t-1} W_{hh}^T + b_{hh}) \tag{2.1}$$

Wie in Gleichung 2.1 gezeigt, beschreibt die Funktion den versteckten Zustand h_t zur Zeit t.

Dabei ist h_t der versteckte Zustand zur Zeit t. Der Term x_t repräsentiert den Input zur Zeit t, während h_{t-1} den versteckten Zustand der vorherigen Schicht zur Zeit $t-1$ oder den initialen versteckten Zustand zur Zeit 0 darstellt. Die Gewichtsmatrix für den Input wird durch W_{ih} dargestellt, und b_{ih} ist der zugehörige Bias. Die Gewichtsmatrix für den versteckten Zustand ist W_{hh}, und b_{hh} ist der Bias für den versteckten Zustand.

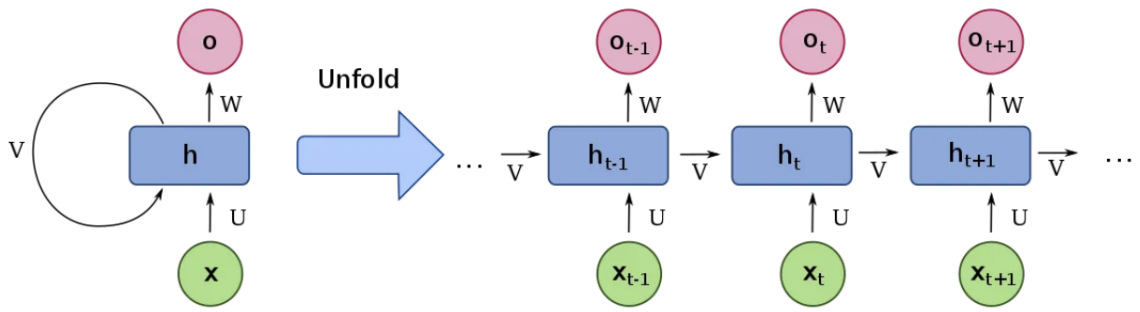

Abbildung 2.1: Eine rekurrente Schicht mit Rückkopplung. Rechts: Die entfaltete Darstellung über mehrere Zeitschritte, die zeigt, wie der vorherige Zustand in den nächsten Schritt weitergegeben wird [Sha24]

In dieser Arbeit wird ein sogenanntes Elman-Netzwerk implementiert. Charakteristisch für das Elman-Netz ist, dass die Ausgänge der ersten versteckten Schicht über Kontextneuronen (den latenten Zustand) mit den Eingangsneuronen rückgekoppelt werden. Für die Implementierung wird das Framework PyTorch verwendet [PyT].

2.2 Transformer

Die Beschreibung des Transformers basiert größtenteils auf dem Paper Attention is All You Need [VSP+17]. In dieser Arbeit wird eine neuartige Architektur namens Transformer vorgestellt, deren charakteristisches Merkmal der Aufmerksamkeitsmechanismus (engl. Attention Mechanism) ist. Die vollständige Architektur ist in Abbildung 2.2 dargestellt. Ähnlich wie rekurrente neuronale Netze (RNNs) dient der Transformer der Transformation einer Sequenz in eine andere und besteht aus zwei Hauptkomponenten: einem Encoder und einem Decoder. In dieser Arbeit wird jedoch ausschließlich der Encoder verwendet.

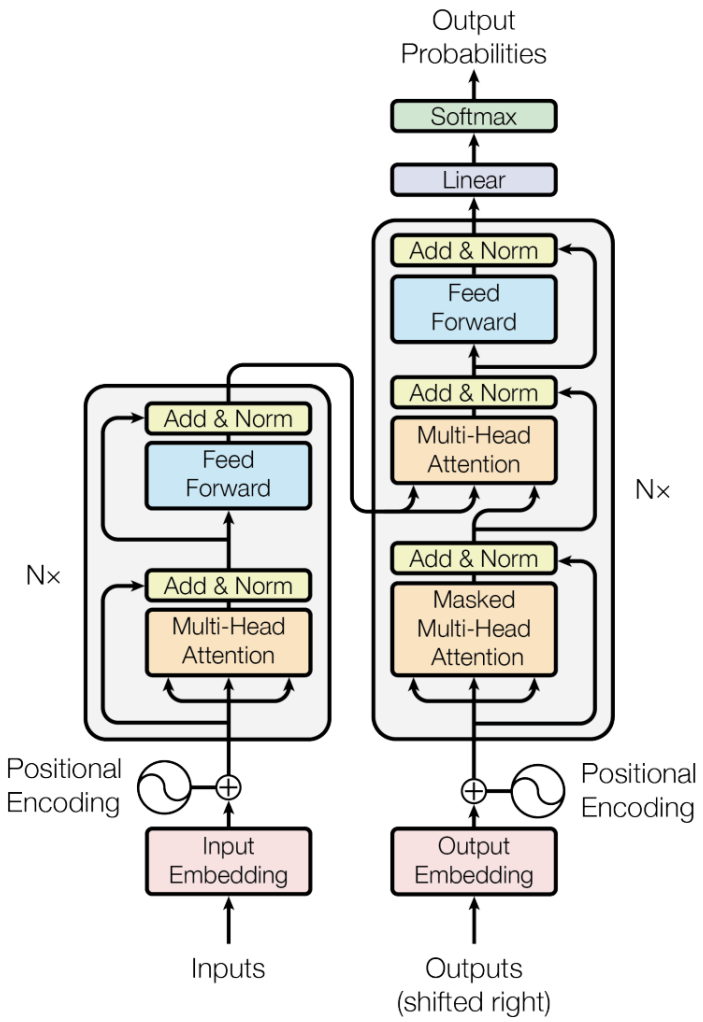

Abbildung 2.2: Transformer-Architektur nach [VSP+17]

Ein wesentlicher Unterschied zu zuvor beschriebenen Architekturen wie RNNs besteht darin, dass der Transformer keine rekurrenten Netze benötigt. Zunächst werden die Schlüsselkomponenten des Modells beschrieben. Der Aufmerksamkeitsmechanismus berechnet einen gewichteten Durchschnitt der Sequenzelemente, wobei die Gewichte dynamisch auf Basis einer Eingabeabfrage (Query) und der Schlüssel (Keys) der Elemente bestimmt werden. Ziel ist es, einen gewichteten Durchschnitt der Merkmale mehrerer Elemente zu bilden, wobei die Gewichtung nicht konstant ist, sondern von den tatsächlichen Werten der Elemente abhängt. Anders ausgedrückt wird dynamisch entschieden, welchen Eingaben mehr Aufmerksamkeit geschenkt wird.
Ein Aufmerksamkeitsmechanismus besteht aus vier zu spezifizierenden Komponenten: Eine Abfrage (Query) beschreibt, wonach in der Sequenz gesucht wird, während ein Schlüssel (Key) angibt, welche Informationen ein Element enthält oder wann es wichtig ist. Werte (Values) sind Merkmale der Eingabeelemente, über die ein ge-

wichteter Durchschnitt gebildet wird. Schließlich wird eine Bewertungsfunktion verwendet, um zu bestimmen, welchen Elementen Aufmerksamkeit geschenkt werden soll. Diese Bewertungsfunktion nutzt die Abfrage und den Schlüssel als Eingabe und gibt eine Gewichtung für jedes Abfrage-Schlüssel-Paar aus. Die endgültigen Gewichtungen werden durch eine Softmax-Normalisierung berechnet, wodurch Werte mit Schlüsseln, die der Abfrage am ähnlichsten sind, eine höhere Gewichtung erhalten. Ein gängiger Ansatz zur Berechnung der Gewichtungen ist die skalierte Skalarprodukt-Aufmerksamkeit (engl. Scaled Dot-Product Attention). Dabei wird das Skalarprodukt zwischen der Abfrage Q und den Schlüsseln K berechnet, anschließend durch die Wurzel der Schlüssel-Dimension d_k skaliert und mit der Softmax-Funktion normalisiert. Schließlich werden die so berechneten Gewichte auf die Werte V angewendet. Mathematisch wird die Dot-Product-Attention wie folgt definiert:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Diese Methode ermöglicht es dem Modell, Abhängigkeiten innerhalb der Sequenz effizient zu erkennen und relevante Informationen zu fokussieren. Da Sequenzelemente oft mehrere verschiedene Aspekte enthalten, reicht ein einzelner gewichteter Durchschnitt nicht aus. Daher wird der Mechanismus durch Multi-Head Attention erweitert, um unterschiedliche Aspekte der Eingabe gleichzeitig zu berücksichtigen. Daher wird der Aufmerksamkeitsmechanismus auf mehrere Köpfe erweitert (*Multi-Head Attention*), indem mehrere unterschiedliche Abfrage-Schlüssel-Wert-Tripel für dieselben Merkmale verwendet werden. Anschließend werden diese Köpfe verkettet und mit einer finalen Gewichtungsmatrix kombiniert, wodurch der Multi-Head-Attention-Mechanismus entsteht 2.3.

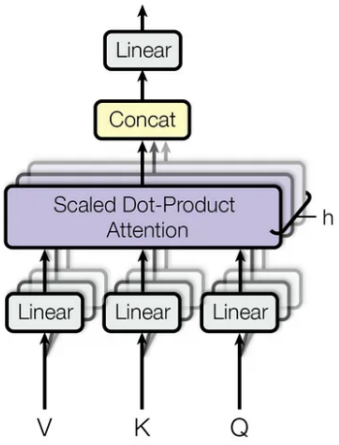

Abbildung 2.3: Multi-Head Attention [VSP+17]

Der Encoder besteht aus mehreren identischen Blöcken, die nacheinander angewendet werden. Zunächst durchläuft die Eingabe einen *Multi-Head Attention*-Block,

wie in Abbildung 2.2 dargestellt. Die Ausgabe wird über eine *Residual Connection* zur ursprünglichen Eingabe addiert, und die resultierende Summe wird einer *Layer Normalization* unterzogen: $\text{LayerNorm}(x + \text{Sublayer}(x))$. Die Residualverbindung ist aus zwei Gründen entscheidend: Erstens wird ein gleichmäßiger Gradientenfluss durch das Modell sichergestellt. Zweitens verhindert sie den Informationsverlust der ursprünglichen Sequenz.
Die Layer-Normalisierung spielt ebenfalls eine zentrale Rolle, da sie das Training stabilisiert und eine gewisse Regularisierung bewirkt. Darüber hinaus wird sichergestellt, dass die Merkmale der Sequenzelemente in ähnlicher Größenordnung bleiben. Zusätzlich zur *Multi-Head Attention* wird ein kleines, vollständig verbundenes Feedforward-Netzwerk in den Encoder integriert, das unabhängig für jede Position angewendet wird. Das Modell nutzt hierfür ein lineares ReLU-MLP.
Da weder Rekurrenz noch Faltung im Modell verwendet werden, müssen Informationen über die Reihenfolge der Sequenz explizit hinzugefügt werden. Dies geschieht durch sogenannte Positionskodierungen (*Positional Encodings*), die an den Eingabeeinbettungen am unteren Ende der Encoder-Stacks ergänzt werden. Diese Kodierungen haben dieselbe Dimension d_{model} wie die Einbettungen, sodass sie direkt summiert werden können.
In dieser Arbeit, ebenso wie in [VSP+17], werden Positionskodierungen auf Basis von Sinus- und Kosinusfunktionen mit unterschiedlichen Frequenzen verwendet:

$$PE(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

$$PE(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

wobei pos die Position und i die Dimension bezeichnet. Jede Dimension der Positionskodierung entspricht somit einer Sinuskurve. Die Wellenlängen dieser Kodierungen folgen einer geometrischen Progression von 2π bis $10000 \cdot 2\pi$.

Die gesamte Codeimplementierung dieser Arbeit basiert auf der Transformer-Implementierung der Webseite *UvA DL Notebooks* [Lip].

2.3 Koopman Operator

Die Beschreibung des Koopman-Operators basiert auf dem Artikel *Modern Koopman Theory for Dynamical Systems* [BBKK22] sowie auf dem Framework, das in dieser Arbeit für die Modellierung verwendet wird, nämlich der Python-Bibliothek *Koopman and Transfer Operator Learning* [NK23].

Die Hauptidee besteht darin, ein endlichdimensionales nichtlineares System durch einen unendlichdimensionalen linearen Operator darzustellen, der auf einem Hilbert-Raum von Messfunktionen des Systemzustands wirkt. Die Bestimmung endlichdimensionaler Matrixnäherungen des Koopman-Operators steht im Mittelpunkt intensiver Forschungsbemühungen und verspricht, globale lineare Darstellungen nichtlinearer dynamischer Systeme zu ermöglichen. Die Darstellung nichtlinearer Dynamik in einem linearen Rahmen ist attraktiv, da eine Vielzahl optimaler Schätz- und Steuerungstechniken für lineare Systeme zur Verfügung steht und der zukünftige Systemzustand analytisch vorhergesagt werden kann.

Gegeben sei eine nichtlineare Funktion $x(t)$. Die Funktionen g werden als Observables bezeichnet. Diese Observables entsprechen den Messungen der nichtlinearen Funktion $x(t)$. Der Koopman-Operator K_t ist ein unendlichdimensionaler linearer Operator, der auf Messfunktionen g wie folgt wirkt:

$$K_t g = g \circ F_t$$

wobei $\circ$ den Kompositionsoperator bezeichnet. Für ein zeitdiskretes System mit einer Zeitschrittweite Δt ergibt sich:

$$K_{\Delta t} g(x_k) = g(F_{\Delta t}(x_k)) = g(x_{k+1})$$

Mit anderen Worten: Der Koopman-Operator definiert ein unendlichdimensionales lineares dynamisches System, das die Beobachtung des Zustands $g_k = g(x_k)$ zum nächsten Zeitschritt fortführt:

$$g(x_{k+1}) = K_{\Delta t} g(x_k)$$

Da dies für jede Observable-Funktion g und jeden Zustand x_k gilt, kann somit im Hilbert-Raum ein Schritt mithilfe des Koopman-Operators berechnet und anschließend wieder in den endlichdimensionalen Raum rekonstruiert werden. Dies ergibt dann die Funktion für den Zeitpunkt x_{k+1}. Die Idee lässt sich mithilfe des Bildes 2.4 nachvollziehen.

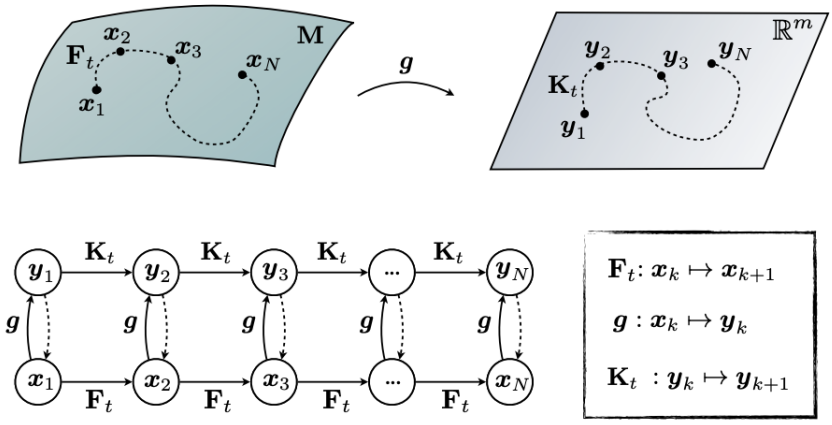

Abbildung 2.4: Schematische Darstellung des Koopman-Operators für nichtlineare dynamische Systeme. Die gestrichelten Linien von y_k nach x_k zeigen an, dass der ursprüngliche Zustand wiederhergestellt werden soll [BBKK22].

Kapitel 3

Systemdatensätze zur Evaluierung der Modelle

Um die in den vorherigen Kapiteln beschriebenen Modellarchitekturen bewerten zu können, wurden vier verschiedene Datensätze zusammengestellt. Die ersten drei basieren auf generierten synthetischen Daten, während der letzte Datensatz reale Messwerte aus einem IoT-Sensorsystem enthält.

3.1 Van-der-Pol-Oszillator

Der Van-der-Pol-Oszillator ist ein nichtlineares, autonomes dynamisches System mit begrenzten Zyklen und besitzt eine stabile Lösung. Das Systemverhalten reicht von nahezu linear (bei geringer Nichtlinearität) bis zu stark nichtlinear (für große Parameterwerte). Die zugehörigen Differentialgleichungen lauten:

$$\dot{x}_0 = ax_1 + \eta_1, \tag{3.1}$$

$$\dot{x}_1 = \mu x_1(1 - bx_0^2) - cx_0 + \eta_2, \tag{3.2}$$

wobei a, μ, b, c Systemparameter sind und η_1, η_2 stochastische Rauschkomponenten darstellen. Für die Experimente wurden die Werte $a = 2$, $\mu = 2$, $b = 5$ und $c = 0.8$ gewählt.

3.2 Van-der-Pol-Oszillator mit Rauschen

Um realistischere Messbedingungen zu simulieren, wird dem Van-der-Pol-Oszillator Gaußsches Rauschen mit einer Standardabweichung von σ hinzugefügt. Dies führt zu den stochastischen Differentialgleichungen:

$$\begin{aligned} \dot{x}_0 &= ax_1 + \eta_1, \\ \dot{x}_1 &= \mu x_1(1 - bx_0^2) - cx_0 + \eta_2, \end{aligned} \tag{3.3}$$

wobei η_1 und η_2 normalverteilte Zufallsgrößen sind, die das Messrauschen modellieren.
Dieser verrauschte Van-der-Pol-Oszillator ermöglicht es, die Robustheit verschiedener Modellierungsansätze zu bewerten, indem untersucht wird, wie empfindlich sie auf störende Einflüsse reagieren.

3.3 Lorenz-System

Das Lorenz-System ist ein berühmtes nichtlineares, deterministisches chaotisches System, das für seine Empfindlichkeit gegenüber Anfangsbedingungen bekannt ist. Die Dynamik wird durch das folgende Gleichungssystem beschrieben:

$$\dot{x} = \sigma(y - x), \tag{3.4}$$
$$\dot{y} = x(\rho - z) - y, \tag{3.5}$$
$$\dot{z} = xy - \beta z. \tag{3.6}$$

Für die Simulation wurden die klassischen Parameterwerte $\sigma = 10$, $\rho = 28$ und $\beta = \frac{8}{3}$ verwendet. Das Lorenz-System zeigt chaotisches Verhalten, was bedeutet, dass kleine Änderungen der Anfangswerte zu drastischen Abweichungen in der Trajektorie führen können. Diese Eigenschaft macht es zu einem herausfordernden Testfall für Modellarchitekturen, die langfristige Vorhersagen treffen sollen.

3.4 Raumtemperatur-Messungen

Der vierte Datensatz besteht aus realen Messungen der Raumtemperatur, die mit IoT-Sensoren erfasst wurden. Die Temperatur ist abhängig von externen Faktoren wie Tageszeit, Heizsystemen und Wetterbedingungen. Dadurch enthält der Datensatz sowohl deterministische als auch stochastische Komponenten.
Dieser reale Datensatz ermöglicht eine praxisnahe Evaluierung der Modellarchitekturen, da er Störfaktoren und nicht ideale Messbedingungen enthält. Die Fähigkeit eines Modells, solche Daten korrekt vorherzusagen, ist entscheidend für Anwendungen in der Gebäudeautomatisierung und Klimaregelung.

Kapitel 4

Ergebnisse

In diesem Abschnitt werden die Modellarchitekturen anhand verschiedener Kriterien verglichen: Implementierungskomplexität (Anzahl der Hyperparameter und Anzahl der Codezeilen), Trainings- und Validierungszeit sowie Testverlust. Dabei ist zu beachten, dass die Architekturen nicht vollständig optimiert implementiert sind und die Hyperparameter nicht als die besten gewählt wurden, sondern lediglich so angepasst wurden, dass die Modelle eine mittlere Leistungsfähigkeit erreichen. Obwohl die Modelle nicht die bestmögliche Struktur besitzen, sollten sie dennoch aussagekräftige Informationen über ihre Eigenschaften liefern, da eine Verbesserung der Architektur die Ergebnisse nicht grundlegend verändern würde.

Zudem ist anzumerken, dass die Datensätze vor dem Training standardisiert wurden. Der Trainings-, Validierungs- und Testdatensatz wurde zentriert und durch die Standardabweichung des Trainingsdatensatzes skaliert. Dadurch wird verhindert, dass sich die Modelle an den Erwartungswert und die Standardabweichung der Validierungs- und Testdaten anpassen, was eine faire Evaluierung sicherstellt.

4.1 Implementierungskomplexität

Bei der Auswertung des Codeumfangs kann es zu kleinen Abweichungen kommen, da auch Kommentare im Code gezählt werden und Abstände zwischen Funktionsblöcken enthalten sind. Dies sollte jedoch den Vergleichsfehler nicht über 10 Prozent beeinflussen. Die Funktionen zur Datenvorbereitung werden hier mit einbezogen.

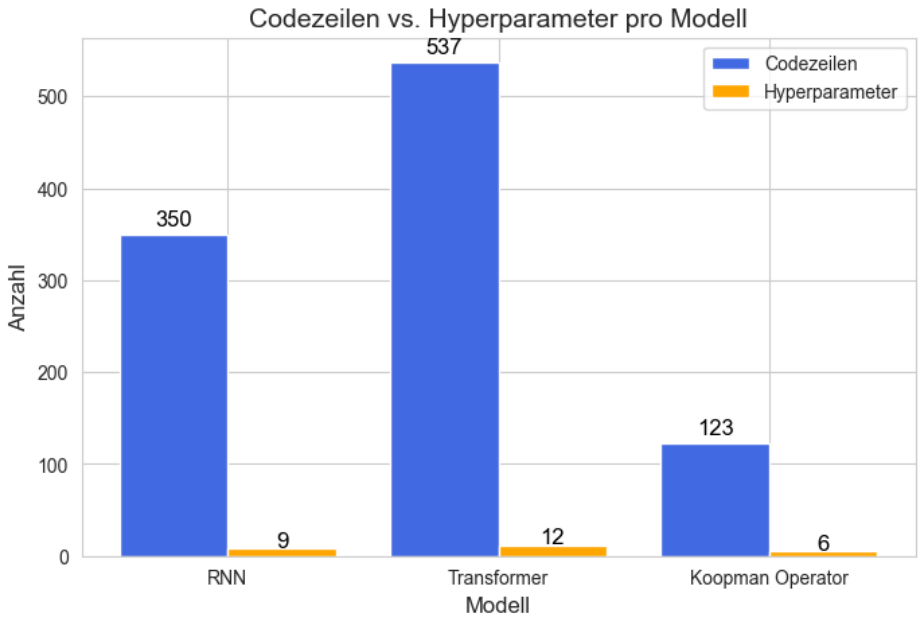

Abbildung 4.1: Vergleich der Implementierungskomplexität: Anzahl der Codezeilen und Hyperparameter für verschiedene Modelle.

Das Verhältnis von Hyperparametern und Codezeilen ist ähnlich und zeigt, dass der Transformer die komplexeste Architektur hat, während der Koopman-Operator am kompaktesten ist. Beim Transformer liegt dies am Self-Attention-Mechanismus, der viele Codezeilen beansprucht. Zudem führt die Kombination aus Attention-Mechanismus, Feedforward-Netzen und Embeddings zu einer hohen Anzahl an Hyperparametern. Der Koopman-Operator hingegen benötigt weniger Code, da er auf bereits im Koopman-Framework integrierte Module zurückgreift.

4.2 Zeitaufwand und Testfehler

Die folgende Grafik zeigt den Testfehler jeder Modellarchitektur für die verschiedenen Datensätze. Zudem ist die Trainingszeit auf der linken y-Achse dargestellt.

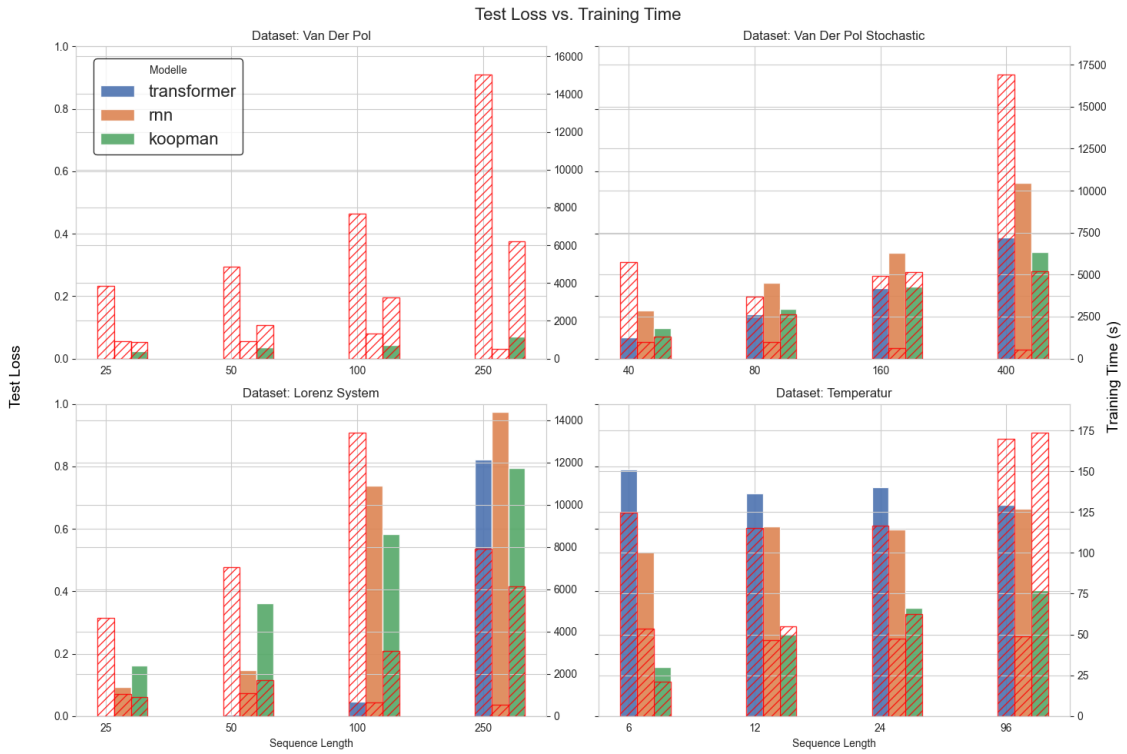

Abbildung 4.2: Vergleich des Testfehler und der Trainingszeit für verschiedene Modelle und Datensätze.

Basierend auf den Ergebnissen zeigt sich, dass sich RNNs als besonders effizient erweisen, insbesondere für den deterministischen Van-der-Pol-Oszillator. Sie benötigen die geringste Trainingszeit und erreichen dennoch eine vergleichbare Genauigkeit zum Transformer. Für verrauschte Systeme, wie den stochastischen Van-der-Pol-Oszillator, schneiden RNNs jedoch schlechter ab, was darauf hindeutet, dass sie mit stochastischer Dynamik weniger gut umgehen können.
Der Transformer zeigt seine Stärken insbesondere bei verrauschten Systemen. Für den stochastischen Van-der-Pol-Oszillator liefert er die besten Fehlerwerte, benötigt allerdings die meiste Rechenzeit. Beim Lorenz-System ist er für kurze Vorhersagen sehr präzise, verliert jedoch bei längeren Sequenzen an Stabilität.
Das Koopman-Modell ist beim deterministischen Van-der-Pol-Oszillator die schlechteste Methode, liefert jedoch bei den IoT-Temperaturdaten die besten Vorhersagen. Auch bei verrauschten Systemen ist es teilweise konkurrenzfähig, insbesondere bei längeren Vorhersagehorizonten. Allerdings nimmt die Trainingszeit für große Sequenzen stark zu.

Kapitel 5

Fazit

Nach dem Vergleich der Testverluste, der Trainingszeit und der Implementierungskomplexität ergibt sich eine differenzierte Bewertung der Modelle. Das RNN ist besonders effizient für deterministische Systeme, während der Transformer bei verrauschten Daten und kurzfristigen Vorhersagen gut performt, jedoch hohe Rechenkosten verursacht. Das Koopman-Modell eignet sich gut für Systeme mit stabilen Strukturen, wie beispielsweise die Raumtemperaturdaten, ist aber für chaotische oder stark nichtlineare Systeme wenig zuverlässig. Die Wahl des optimalen Modells hängt daher stark von den Eigenschaften des zugrunde liegenden Systems und den verfügbaren Rechenressourcen ab.
Der gesamte Code und die Implementierung dieser Studie sind auf GitHub verfügbar unter: `https://github.com/marej9/Modeling_Dynamic_Systems`GitHub-Repository

Abbildungsverzeichnis

2.1 Eine rekurrente Schicht mit Rückkopplung. Rechts: Die entfaltete Darstellung über mehrere Zeitschritte, die zeigt, wie der vorherige Zustand in den nächsten Schritt weitergegeben wird [Sha24] 8
2.2 Transformer-Architektur nach [VSP+17] 9
2.3 Multi-Head Attention [VSP+17] 10
2.4 Schematische Darstellung des Koopman-Operators für nichtlineare dynamische Systeme. Die gestrichelten Linien von y_k nach x_k zeigen an, dass der ursprüngliche Zustand wiederhergestellt werden soll [BBKK22]. 13

4.1 Vergleich der Implementierungskomplexität: Anzahl der Codezeilen und Hyperparameter für verschiedene Modelle. 18
4.2 Vergleich des Testfehler und der Trainingszeit für verschiedene Modelle und Datensätze. 19

Tabellenverzeichnis

Literaturverzeichnis

[BBKK22] Steven L. Brunton, Marko Budisic, Eurika Kaiser, and J. Nathan Kutz. Modern koopman theory for dynamical systems. *SIAM Review*, 64(2):229–340, 2022. Published under the terms of the Creative Commons 4.0 license. doi:10.1137/20M1353171.

[End24] Shristian Endisch. *Künstliche Neuronale Netze zur Identifikation mechatronischer Systeme*. Technische Universität München, 2024. Wintersemester 2024/25.

[Lip] Phillip Lippe. Tutorial 6 (jax): Transformers and multi-head attention. URL: https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/JAX/tutorial6/Transformers_and_MHAttention.html.

[NK23] Pietro Novelli and Vladimir Kostic. Kooplearn, 2023. Zugriff am 18. März 2025. URL: https://kooplearn.readthedocs.io/latest/primer.html.

[Pat23] Omkar Patil. Time series forecasting with machine learning, 2023. Zugriff am 18. März 2025. URL: https://medium.com/@omkar.patil20/time-series-forecasting-with-machine-learning-68dbcd1ccb0d.

[PyT] PyTorch. Rnn. URL: https://pytorch.org/docs/stable/generated/torch.nn.RNN.html.

[Sha24] Hadar Sharvit. What is a recurrent neural network — elman net, 2024. Zugriff am 18. März 2025. URL: https://medium.com/@hadarsharvit/what-is-a-recurrent-neural-network-elman-net-part-1-d9a70d8e0b95.

[VSP+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st Conference on Neural Information Processing Systems (NIPS 2017)*, pages 5998–6008, Long Beach, CA, USA, 2017. doi:10.5555/3295222.3295349.

License

This work is licensed under the Creative Commons Attribution 3.0 Germany License. To view a copy of this license, visit http://creativecommons.org or send a letter to Creative Commons, 171 Second Street, Suite 300, San Francisco, California 94105, USA.